

Gravity Lens — Full Statement

Overview

Gravitational lensing is a phenomenon where light from a distant source is bent by the gravitational field of a massive object (the lens) as it passes by. This effect distorts the appearance of the source, creating magnified, sheared, or multiple images. In this simplified simulation, you are given a 64×64 grayscale image of a lensed system. The lensing is modeled by a 2D Gaussian convolution kernel. The kernel's width parameters depend on two physical quantities:

- **Mass** (M) – a continuous value in $[1.0, 10.0]$ that scales the overall blur.
- **Ellipticity** (e) – a binary label: 0 for a spherical lens (circular kernel), 1 for an elliptical lens (anisotropic kernel).

Your task is to build a model that, given an image, predicts both the mass (regression) and the ellipticity class (classification). The dataset is artificial, but it mimics the challenge of recovering physical parameters from noisy observations.

Physics Background

The lensing effect is approximated by convolving a random source pattern $S(x, y)$ with a 2D Gaussian kernel $K(x, y)$:

$$I(x, y) = (S * K)(x, y) + \mathcal{N}(0, \sigma_n^2)$$

where $\sigma_n = 0.02$ is the standard deviation of additive Gaussian noise. The kernel is defined as:

$$K(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} \exp\left(-\frac{x^2}{2\sigma_x^2} - \frac{y^2}{2\sigma_y^2}\right)$$

The widths σ_x and σ_y are linked to mass and ellipticity by:

$$\sigma_x = M \cdot (1 + e), \quad \sigma_y = M \cdot (1 - e)$$

- When $e = 0$, $\sigma_x = \sigma_y = M$ – spherical kernel.
- When $e = 1$, $\sigma_x = 2M$, $\sigma_y = 0$ – extremely elongated (elliptical).

The source pattern S consists of uniform background plus a few random point sources, ensuring each image is different.

Dataset Description

You are provided with the following files:

- `train_images.npy` – shape `(1000, 1, 64, 64)`, `float32` – training images.
- `train_labels_mass.npy` – shape `(1000,)`, `float32` – true mass values for training images.
- `train_labels_ellip.npy` – shape `(1000,)`, `int64` – true ellipticity labels (0 or 1) for training images.
- `test_images.npy` – shape `(300, 1, 64, 64)`, `float32` – test images (no labels provided).

All images are grayscale and have pixel values in $[0, 1]$. The training set contains 1000 samples; the test set contains 300 samples.

Subtasks

Subtask A: Mass Regression (50 points)

Predict the continuous mass M for each test image.

Metric: Root Mean Squared Error (RMSE).

Scoring:

- $\text{RMSE} \leq 1.0 \rightarrow 50$ points
- $\text{RMSE} \geq 4.0 \rightarrow 0$ points
- Linear interpolation between 1.0 and 4.0.

Subtask B: Ellipticity Classification (50 points)

Predict the binary class (0 = spherical, 1 = elliptical) for each test image.

Metric: Accuracy.

Scoring:

- $\text{Accuracy} \geq 0.95 \rightarrow 50$ points
- $\text{Accuracy} \leq 0.50 \rightarrow 0$ points
- Linear interpolation between 0.50 and 0.95.

Submission Format

You must submit a single pickle file `submission.pkl` containing an object with the following methods:

python

 Copy  Download

```
class Submission:
    def predict_A(self, images: np.ndarray) -> np.ndarray:
        # images: shape (N, 1, 64, 64), dtype float32
        # returns: array of predicted masses, shape (N,), dtype float32
        pass

    def predict_B(self, images: np.ndarray) -> np.ndarray:
        # returns: array of predicted classes (0 or 1), shape (N,), dtype int64
        pass
```

The grader will load the object, call these methods on the test images, and compute the scores. Your submission must be serialized with `cloudpickle` (as shown in the starter notebook).

Constraints

- Runtime: ≤ 10 minutes on a free GPU (T4 / P100).
- No pre-trained models (ResNet, ViT, etc.) – you must train your own model from scratch.
- No external data beyond the provided files.
- Your `submission.pkl` must be ≤ 50 MB.

Starter Kit

A starter notebook (`gravity_lens_starter.ipynb`) is provided that:

- Loads the data.
- Defines a baseline CNN with two output heads.
- Trains the model and saves a submission file.
- Implements the required `Submission` class with placeholder prediction methods that you can improve.

You are free to modify any part of the code to achieve better performance.

Files in the Starter Kit

- `gravity_lens_starter.ipynb` – the main notebook.
 - `train_images.npy`, `train_labels_mass.npy`, `train_labels_ellip.npy`, `test_images.npy` – the dataset.
 - `gravity_lens_solution.ipynb` – a full solution (optional reference).
-

Example of a Correct Submission

python

 Copy  Download

```
import cloudpickle
import numpy as np

class Submission:
    def predict_A(self, images):
        # Dummy: return random masses
        return np.random.uniform(1.0, 10.0, size=len(images))
    def predict_B(self, images):
        return np.random.randint(0, 2, size=len(images))

sol = Submission()
with open('submission.pkl', 'wb') as f:
    cloudpickle.dump(sol, f)
```